SEN381MILESTONE 2

Person 1Riegardt Christopher Weich 602547
Person 2 Ditheto Lesiba Phaho 601133
Person 3 Armand Johannes Vermeulen 602636

CivicConnect — PED v2.0

Document control and M1 baseline review (PED v2.0)

Version history

Version	Date	Author	Changes
0.1	06 September	Armand	Work split and format *(M1, retained)*
0.8	08 September	Team	Each member integrates sections *(M1, retained)*
1.0	09 September	Team	Verified M1 baseline for submission *(M1, retained)*
1.1	09 September	Person 1	M1 review findings logged as change requests CR-001 to CR-003 (Section 1.2)
2.0	10 September	Team	Architecture, Technology & Initial Design Baseline: ASRs, architecture decision, ADR-001 to ADR-004, RTM evolution, updated risks and forward considerations

M1 baseline review: findings requiring controlled change

Reviewing M1 against what architecture now needs surfaced three issues. They touch baselined content, so they are raised as change requests, not silently edited (the M2 brief requires the original baseline and reason for change to be preserved).

CR	M1 item	Issue	Proposed change	Reason
CR-001	NFR-024 / AC-024	Requirement says "at least 95-98% of requests" but the acceptance criterion says 95%. The range is ambiguous and cannot be tested.	Restate as: "at least 95% of regular operations return feedback within 3 seconds under the expected load [define load]." Original wording retained in the change log.	A testable target is needed to evaluate architecture against ASR-5.

CR	M1 item	Issue	Proposed change	Reason
CR-002	FR-015	"Overdue" is required in reporting but M1 defines no due date or overdue rule (SLA escalation is deferred in Section 3.3).	Add a clarification: a request is overdue when [rule, e.g. unresolved after N working days by category]. Escalation remains deferred.	Overdue status needs a data attribute (due date / threshold). This affects the data model (Person 2) and the Reporting module.
CR-003	AC-0010 (FR-010)	Identifier typo; all others use three digits (AC-001 to AC-016).	Correct to AC-010.	Traceability integrity.

Assumptions and constraints carried forward

The M1 constraints (scope, schedule, cost, quality, security) remain in force. Two are now architecturally active:

- **Cost/resources:** M1 requires free-tier limits to be recorded as an explicit assumption. Architecturally this means a single deployable unit on a single host is the default assumption, to be revisited with the technology and deployment evidence (Person 2).
- **Security/POPIA:** M1 requires personal information to be handled lawfully under POPIA (Republic of South Africa, 2013). This becomes ASR-1 below.

Architecture Significant Requirements (ASRs) and quality drivers

An ASR is a requirement that has a measurable effect on the architecture: if it changed, the structure of the system would have to change (Bass, Clements and Kazman, 2021). Not all 24 M1 requirements qualify. Five do.

Selected ASRs

ASR	Quality attribute (ISO/IEC, 2011)	Driven by (M1 evidence)	Measurable expectation	Architectural consequence
ASR-1 Controlled	Security: confidentiality, authenticity	NFR-017, NFR-018, NFR-019, FR-007; Security	100% of protected operations are denied to	Authorisation is enforced on the server in one central place

ASR	Quality attribute (ISO/IEC, 2011)	Driven by (M1 evidence)	Measurable expectation	Architectural consequence
access to request data		constraint; Management-vs-privacy conflict (M1 §2.1); POPIA	unauthorised roles in automated tests; no data-returning endpoint lacks an authorisation check.	(deny by default), not in the UI and not scattered through feature code.
ASR-2 Trustworthy request lifecycle and audit trail	Security: accountability/non-repudiation; Functional correctness	NFR-020, FR-011, FR-012, FR-013; Management accountability need	Every status or ownership change produces exactly one audit record, written in the same transaction as the change; invalid status transitions are rejected 100% of the time in tests.	The lifecycle rules live in one domain component and the audit write is not optional or separable from the change.
ASR-3 No lost requests	Reliability: recoverability, fault tolerance	NFR-022, FR-003; Requester need "confidence the request will not be lost"	A request is persisted before confirmation is shown; 100% of confirmed requests retrievable by ID after a simulated application restart.	Persist-then-confirm flow; a durable relational store is the system of record; backup approach is a baseline concern (Person 2 owns the data decision).

ASR	Quality attribute (ISO/IEC, 2011)	Driven by (M1 evidence)	Measurable expectation	Architectural consequence
ASR-4 Modifiability by a small team	Maintainability: modularity, testability	NFR-023; Schedule constraint; deferred scope (SLA escalation, SMS/email, analytics)	Adding a new notification channel or a new report touches one module plus configuration (verified by design review against the module map); each module is testable without the others running.	Enforced module boundaries with explicit interfaces; extension points where M1 deferred features will land.
ASR-5 Responsive everyday operations	Performance efficiency: time behaviour	NFR-024 (as clarified by CR-001); FR-008, FR-015	At least 95% of regular operations give feedback within 3 s under [expected load].	Filtering, searching and reporting must be served by indexed queries, not by loading and filtering in application memory.

Considered and deliberately not selected as ASRs

Naming what is excluded keeps this list from becoming a generic catalogue of every quality attribute taught.

- **Usability (NFR-021):** important, but it is satisfied in the presentation layer and does not change system structure.
- **Scalability beyond a single instance:** M1 lists scalability as a forward consideration but supplies no expected user volumes (its own "missing information"). Without a number, designing for horizontal scale would be unjustified complexity. It is recorded as a deferred decision (Section 8).

- **Availability (e.g. 99.9%):** no availability target exists in M1. Recorded as a risk and deferred decision, not an invented ASR.

How the ASRs drive decisions across the baseline

ASR	Architecture (Person 1)	Data (Person 2)	Technology (Person 2)	Design (Person 3)
ASR-1	Central authorisation enforcement (ADR-003)	Ownership attributes on requests so access rules can be evaluated	Stack must support server-side role checks and sessions	Interface/guard approach
ASR-2	Lifecycle in one component, audit co-located in transaction (ADR-004)	Append-only audit entity; transactional writes	Store must support ACID transactions	State/transition pattern candidate
ASR-3	Persist-then-confirm; single system of record	Durable relational model, backup/recovery	Mature relational DB	Validation and unit-of-work approach
ASR-4	Modular monolith with dependency rules (ADR-001, ADR-002)	Per-module data ownership	Build/test tooling per module	Notification extension point
ASR-5	Query-side separation for reporting (within the monolith)	Indexes on filter attributes	Query capability of the store	Pagination/filter approach

Column entries for Persons 2 and 3 are proposals to align on; they should confirm or overwrite.

Architecture alternatives and decision

Alternatives considered

Option	Description
A. Layered monolith	One deployable, organised only by technical layers (presentation, business, data). The classic layered style (Richards and Ford, 2020).
B. Modular monolith	One deployable, organised by business modules with explicit interfaces and dependency rules, layered internally (Richards and Ford, 2020; Fowler, 2015).
C. Microservices	Independently deployed services per business capability, communicating over the network (Newman, 2021).
D. Backend-as-a-service / serverless-first	Managed platform supplies auth, database and functions; custom logic is thin and vendor-hosted.

Evaluation against drivers

Scores 1 (poor) to 5 (strong). Weights reflect M1 priorities: security and accountability were the drivers M1 explicitly protected in its Schedule-versus-Quality trade-off (M1 §4.1). **These scores are team judgement, not measurement; [confirm or adjust as a team before sign-off, since you must be able to defend each one].**

Criterion	Weight	A	B	C	D
ASR-1 Access control enforceable consistently	20	4	4	3	3
ASR-2 Atomic lifecycle + audit	20	5	5	2	3
ASR-3 Reliability (fewer failure points)	10	4	4	3	4
ASR-4 Modifiability	15	2	4	4	2
ASR-5 Performance	5	4	4	3	4
Team capability / schedule	15	5	4	1	4
Cost and deployability (free-tier assumption)	15	5	5	1	4
Weighted score (out of 5)	100	**4.20**	**4.35**	**2.35**	**3.30**

Reasoning behind the notable scores:

- **C scores low on ASR-2:** a status change and its audit record would span service boundaries, which forces either distributed transactions or eventual consistency. Both work against "exactly one audit record per change" (Newman, 2021).
- **C scores lowest on team and cost:** a three-person team carrying multiple deployables, inter-service networking and deployment pipelines is disproportionate. The M2 brief states that a more distributed architecture is not automatically more advanced.
- **D scores lower on ASR-1/ASR-2/ASR-4:** authorisation and audit rules would live in vendor-specific configuration, and M1's cost constraint already requires free-tier limits to be treated as a risk. [Person 2: confirm against the technology comparison.]
- **A vs B is close (4.20 vs 4.35).** The deciding factor is ASR-4: M1 has three deferred features (SLA escalation, external notifications, analytics) that need somewhere to land. Plain layers let business logic sprawl across features; module boundaries contain it, at the cost of discipline and a few extra interfaces.

Decision

Modular monolith (Option B): one deployable application, structured as business modules, each internally layered. See ADR-001. The rejected alternative to be ready to defend in the presentation is **C (microservices)**.

Logical architecture

Modules and responsibilities

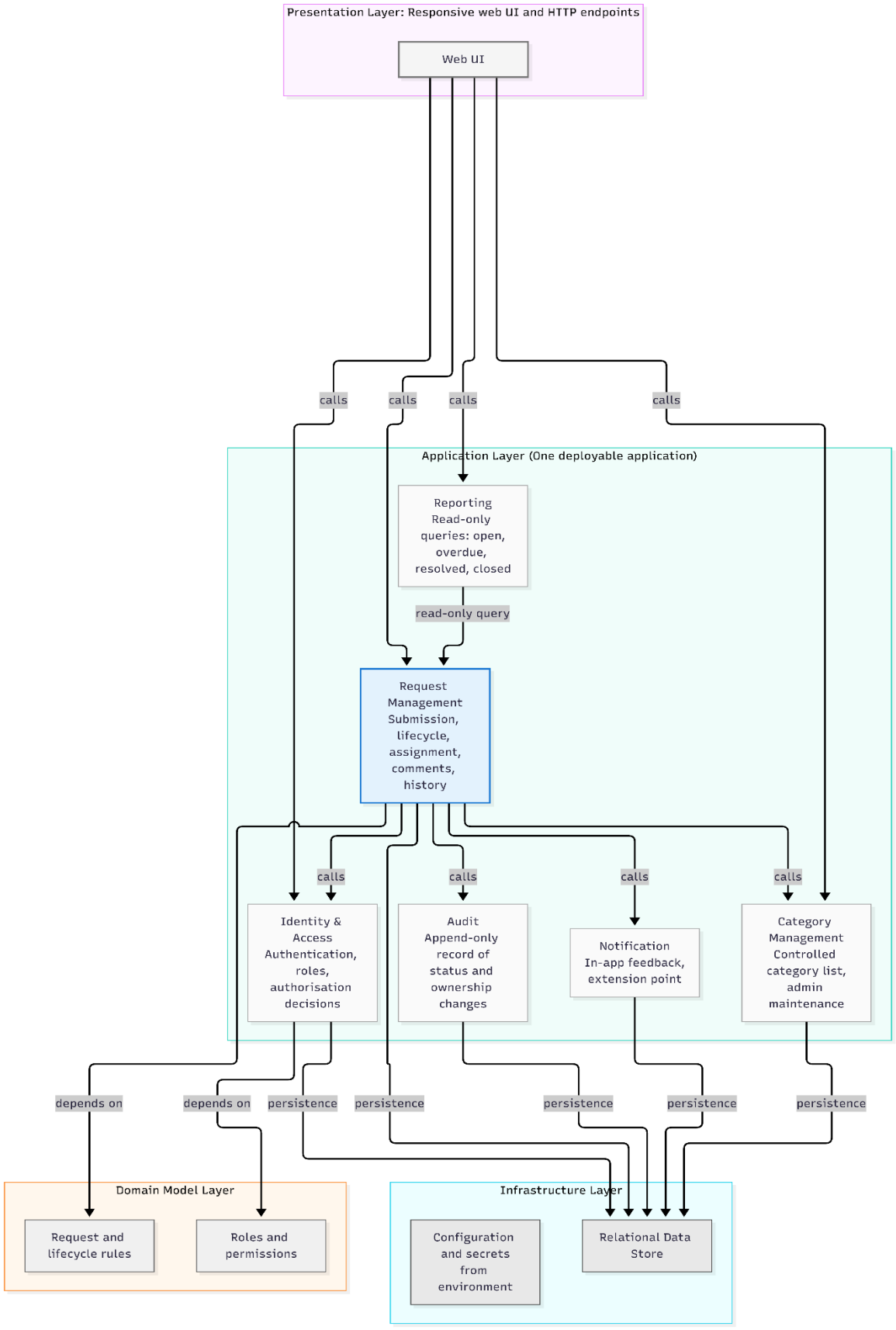
Module	**Responsibility**	**Main M1 requirements**
Identity & Access	Authentication, roles (Requester, Staff, Management/Admin), central authorisation decisions	NFR-017, NFR-018, NFR-019, FR-007
Request Management *(core)*	Submission, unique ID, lifecycle state rules, ownership/assignment, comments and resolution notes, request history views	FR-001, 003, 004, 005, 007 to 013

Module	Responsibility	Main M1 requirements
Category Management	Controlled category list; admin maintenance; validation that a request uses a valid category	FR-002, System Admin need
Audit	Append-only record of status, ownership and other major changes	NFR-020, FR-012, FR-014
Notification	In-app status feedback to requesters; extension point for deferred channels	FR-006
Reporting	Read-side queries for open, overdue, resolved, closed requests by category and status	FR-014, 015, 016

Module dependency rules (recorded in ADR-002):

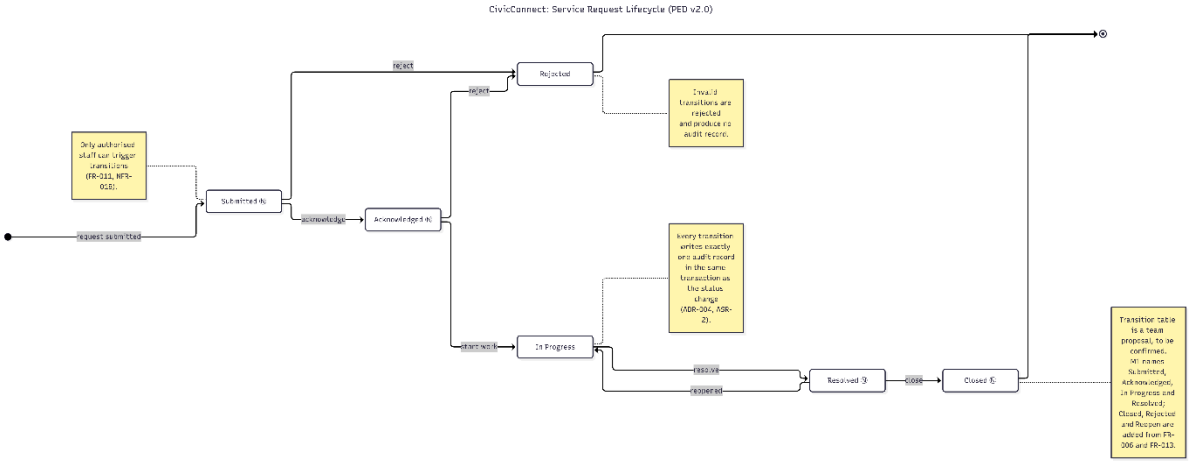
1. Modules talk to each other only through their published interface, never by reaching into another module's internals or tables.
2. Dependencies point inward: presentation → application → domain; infrastructure implements interfaces defined by the inner layers (Martin, 2017).
3. Request Management may call Audit, Notification, Identity & Access and Category Management. None of those may call Request Management. (Prevents circular dependencies.)
4. Reporting is read-only and may read Request Management data through a query interface.

Logical diagram (Mermaid source; redraw in your diagramming tool of choice if preferred)



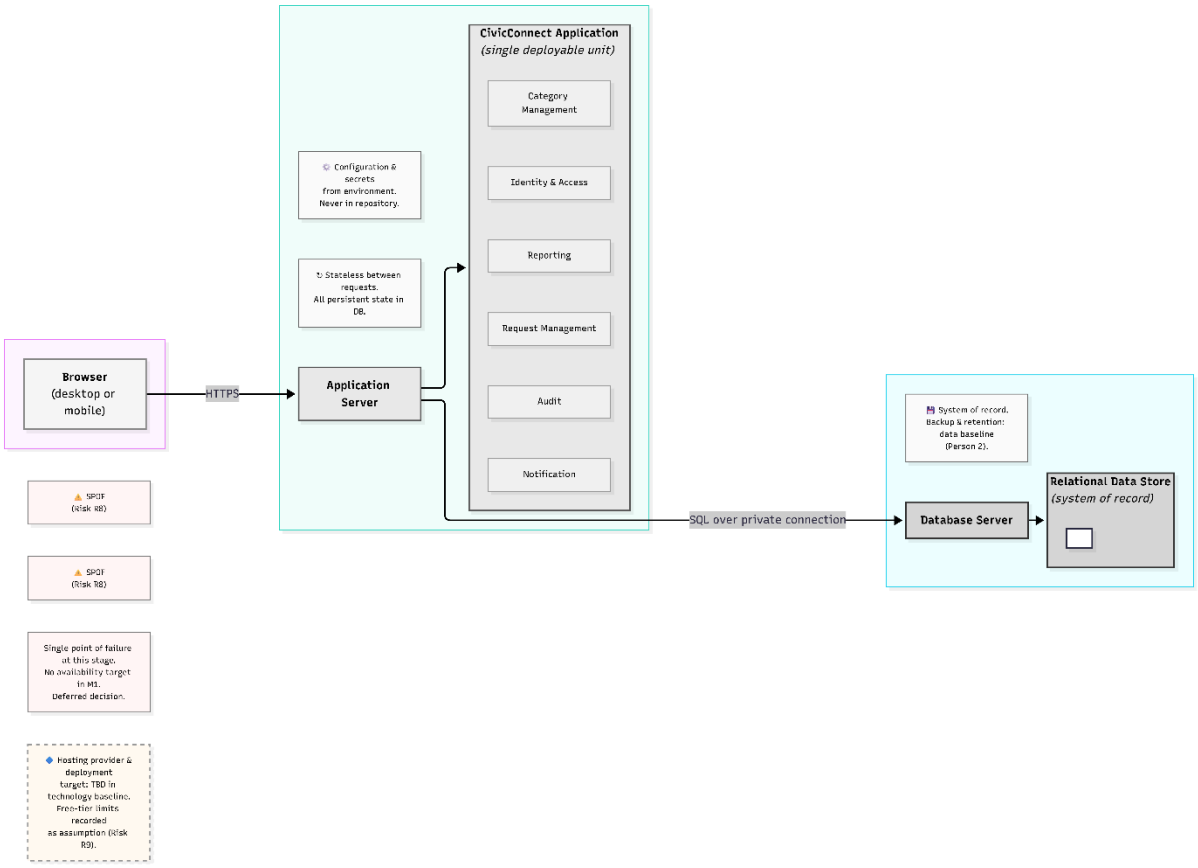
Request lifecycle (the rule enforced in one place, ASR-2)

M1 names the transitions Submitted → Acknowledged → In Progress → Resolved (M1 §2.1) and FR-006/FR-013 add rejected and closed outcomes. The exact transition table below is a **proposal to confirm with the team**, because M1 does not define it fully.



Physical deployment view (deliberately separate from the logical view)

The logical view above says how the code is organised. This view says where it runs. They are different things: several logical modules share one physical process (Kruchten, 1995; Brown, n.d.).



- **Three physical tiers:** browser, application server, database. One deployable application.
- **Deployment target, hosting provider and exact technologies:** not decided here. They belong to the technology baseline (Person 2) and the deployment compatibility section, and must be plausible against M1's free-tier assumption. The architecture is written to be technology-neutral so the A1 candidate stacks remain valid options.
- **Known implications recorded:** configuration and secrets come from the environment and are never committed; the database holds all persistent state, so the application tier stays stateless between requests; one application instance and one database are single points of failure at this stage (Risk R8).

Architecture Decision Records

Format adapted from Nygard (2011). Status values: Proposed / Accepted / Superseded. All four are **Proposed** until the team signs the baseline.

ADR-001: Modular monolith as the architectural style

- **Status:** Proposed | **Date:** [date] | **Owner:** Person 1
- **Context:** Three-person team, fixed milestones, free-tier cost assumption, and five ASRs dominated by security, atomic audit and modifiability (Section 2). Three M1 features are deferred and must be addable later.
- **Decision:** One deployable application structured as six business modules with enforced boundaries and internal layering (Section 4).
- **Alternatives considered:** A layered monolith (little protection against sprawl), C microservices (breaks atomic change-plus-audit, disproportionate operational load), D BaaS-first (rules held in vendor configuration, free-tier and lock-in risk). Scores in Section 3.2.
- **Consequences:** (+) transactions span modules, simple deployment, cheap to run, easy to test in-process. (−) modules share one deployment and one failure domain; boundaries depend on team discipline and code review rather than the network, so violations must be caught in PR review; scaling is whole-application.
- **Evidence:** Bass, Clements and Kazman (2021); Richards and Ford (2020); Fowler (2015); Newman (2021). [Week 2 teaching reference: insert lecture/slide.] [A2 reference if architecture/SCM research informed this: insert section.]
- **If changed, affects:** Section 4 diagrams, RTM module column, deployment assumptions, ADR-002 to ADR-004, risks R5 to R8.

- **Revisit trigger:** measured load breaching ASR-5, or an expected-volume figure that a single instance cannot serve.

ADR-002: Module decomposition and dependency rules

- **Status:** Proposed | **Date:** 30/9/2026 | **Owner:** Person 1
- **Context:** ASR-4 requires change isolation; NFR-023 requires well-defined component responsibilities.
- **Decision:** The six modules and four dependency rules in Section 4.1. Boundaries follow business capabilities rather than technical layers (Evans, 2004).
- **Alternatives considered:** Two or three coarse modules (less overhead, less isolation); one module per M1 requirement group (too granular for the team size).
- **Consequences:** (+) clear ownership and testability. (−) extra interfaces to maintain; Audit and Notification are small modules that may look over-split. Justified by their independent reasons to change (audit rules vs. future channels).
- **Enforcement:** folder/package structure mirrors modules; PR review checklist includes "no cross-module internals"; [optional: automated dependency check if the stack supports it, recorded once Person 2 confirms the stack].
- **Evidence:** Martin (2017); Evans (2004); Bass, Clements and Kazman (2021).
- **If changed, affects:** repository structure, README module map, RTM allocation column.

ADR-003: Central, deny-by-default server-side authorisation

- **Status:** Proposed | **Date:** 30/9/2026 | **Owner:** Person 1
- **Context:** ASR-1. Request data may be sensitive and must be protected under POPIA (Republic of South Africa, 2013). M1 selected role-based access control following the NIST/ANSI model (Sandhu, Ferraiolo and Kuhn, 2000; ANSI/INCITS, 2004). Broken access control is the most common web application weakness in the OWASP ranking (OWASP, 2021).
- **Decision:** Identity & Access is the only module that makes access decisions. Every protected operation passes through it on the server, and the default outcome is deny. UI hiding of controls is a usability aid only, never the control itself. Role checks are combined with ownership checks (e.g. a requester sees only their own requests).

- **Alternatives considered:** Per-feature ad hoc checks (inconsistent, hard to audit); UI-only restrictions (not a control at all); delegating entirely to a vendor policy layer (see ADR-001, option D).
- **Consequences:** (+) one place to review and test; matches the AC-017 to AC-019 acceptance evidence. (−) every new feature must route through it, and a bug there has wide impact, so it needs the strongest automated tests.
- **Evidence:** Sandhu, Ferraiolo and Kuhn (2000); ANSI/INCITS (2004); OWASP (2021); Republic of South Africa (2013), section 19 [verify section wording against the Act before submission].
- **If changed, affects:** NFR-017 to NFR-019 traceability, Identity & Access interface, every endpoint's test plan.

ADR-004: Lifecycle rules and audit record written in one transaction

- **Status:** Proposed | **Date:** 30/9/2026 | **Owner:** Person 1 (co-owned with Person 2 for the data aspects)
- **Context:** ASR-2 and ASR-3. Management needs reliable accountability data (M1 §1.3); the audit trail must not be able to drift from the actual state of a request.
- **Decision:** Status and ownership changes are applied only through Request Management's lifecycle rules (Section 4.3). The audit entry is written in the same database transaction as the change, so both succeed or neither does. Audit records are append-only: no edit or delete path is exposed.
- **Alternatives considered:** Audit written afterwards by a separate process or event (can lose entries on failure); database triggers only (logic hidden from code review, harder to test); no separate audit store, derived from current state only (loses history).
- **Consequences:** (+) audit integrity is structural, not procedural. (−) audit writes add a small cost to each change (acceptable against ASR-5); audit tables grow over time, so retention is an open data decision.
- **Evidence:** Bass, Clements and Kazman (2021); Fowler (2002) on domain logic organisation; [A2 persistence/transaction research section if used: insert reference].
- **If changed, affects:** data model (Person 2), NFR-020 traceability, Reporting (FR-014).

ADR candidate for the team to decide ownership of (overlaps Person 3's design decisions): in-app notification mechanism for FR-006, and how the extension point for

deferred channels is shaped. Record as ADR-005 once Person 3's design patterns are final.

Risk Register additions (architecture-related)

Continues the M1 register (R1 to R4) using the same columns. Person 3 owns the register; these are submitted for merge.

ID	Risk	Probability	Impact	Priority	Mitigation	Owner	Contingency	Status
R5	Module boundaries erode inside the monolith (shortcuts across modules)	Medium	Medium	Medium	ADR-002 rules, PR checklist item, folder structure mirrors modules	Architecture lead	Refactoring task scheduled before the next milestone	Open
R6	Access control bypassed through an endpoint that skips the central check	Medium	High	High	ADR-003; authorisation tests per endpoint; review checklist	Team	Security review and fix before release; log as change	Open
R7	Architecture over-engineered relative to the team's capacity	Low	Medium	Low	Proportionality principle (ADR-001); revisit if a module has no independent reason to exist	Architecture lead	Merge modules via a controlled ADR change	Open

ID	Risk	Probability	Impact	Priority	Mitigation	Owner	Contingency	Status
R8	Single application instance and single database are single points of failure; no availability target defined	Medium	Medium	Medium	Persist-then-confirm (ASR-3); backup approach decided by Person 2; record availability as a deferred decision	Team	Restore from backup; document recovery steps	Open
R9	Free-tier hosting limits conflict with ASR-5 or ASR-3	Medium	Medium	Medium	Record limits as a dependency assumption (M1 Cost constraint); measure early	Person 2	Move to a paid tier or simplify reporting queries	Open

---

RTM: architecture columns (for M2 merge)

Adds the architecture columns to the M1 RTM. The M1 RTM grouped requirements by stakeholder; the per-requirement rows below are a finer-grained view suitable for tracing. Data, design, technology, implementation and verification columns are for Persons 2 and 3; **use the controlled status "Planned" where evidence does not yet exist, and do not back-fill evidence that has not happened.**

Req ID	ASR link	Architecture module(s)	ADR / risk ref
FR-001	ASR-3	Request Management; Category Management (validation)	ADR-001, ADR-004
FR-002	ASR-4	Category Management	ADR-002
FR-003	ASR-3	Request Management	ADR-004
FR-004	ASR-1	Request Management; Identity & Access	ADR-003

Req ID	ASR link	Architecture module(s)	ADR / risk ref
FR-005	ASR-1, ASR-2	Request Management; Audit	ADR-003, ADR-004
FR-006	ASR-4	Notification	ADR-002 (ADR-005 candidate)
FR-007	ASR-1	Request Management; Identity & Access	ADR-003, R6
FR-008	ASR-5	Request Management (query)	ADR-002
FR-009	ASR-1	Request Management; Identity & Access	ADR-003
FR-010	ASR-1, ASR-2	Request Management; Identity & Access; Audit	ADR-003, ADR-004
FR-011	ASR-2	Request Management (lifecycle); Audit	ADR-004
FR-012	ASR-1, ASR-2	Request Management; Audit	ADR-003, ADR-004
FR-013	ASR-2	Request Management (lifecycle); Audit	ADR-004
FR-014	ASR-1, ASR-2	Reporting; Audit	ADR-003, ADR-004
FR-015	ASR-5	Reporting	CR-002
FR-016	ASR-5	Reporting	ADR-002
NFR-017	ASR-1	Identity & Access	ADR-003
NFR-018	ASR-1	Identity & Access	ADR-003
NFR-019	ASR-1	Identity & Access; Request Management	ADR-003, R6
NFR-020	ASR-2	Audit	ADR-004
NFR-021	none (not an ASR)	Presentation layer	n/a

Req ID	ASR link	Architecture module(s)	ADR / risk ref
NFR-022	ASR-3	Infrastructure (data store)	ADR-004, R8
NFR-023	ASR-4	All modules	ADR-001, ADR-002, R5
NFR-024	ASR-5	Reporting; Request Management (query)	CR-001, R9

Suggested end-to-end trace (required by the M2 brief, Section 9)

Pick one path the team will actually implement first. A strong candidate because it touches every ASR decision:

FR-011 (update status through controlled transitions) → ASR-2 → Request Management lifecycle rules + Audit → audit entity and transaction (Person 2) → state/transition design decision (Person 3) → technology and ADR-004 → [branch/class/endpoint] → [test showing an invalid transition is rejected and a valid one writes one audit record].

Fill the bracketed items only with real repository evidence. The brief explicitly penalises decisions with no visible application translation once implementation has begun.

Forward Engineering Considerations (updates) and deferred decisions

M1's six considerations are retained. Updates from architecture:

Consideration	Update in v2.0
Security	Now concrete: ADR-003. Still missing: exact authentication mechanism and session handling (Person 2 technology decision).
Scalability	Single instance assumed (ADR-001). Still missing: expected user volumes. **Deferred** until a volume estimate exists.
Testability	Modules testable in isolation (ASR-4). Test environment definition still pending.
Deployment	Logical vs physical views separated (Section 4.4). Hosting platform deferred to Person 2's deployment section.

Consideration Update in v2.0

Data recovery	Persist-then-confirm adopted (ASR-3). Retention and backup frequency remain open (Person 2).
Observability	Still deferred. No ADR in M2.

Deliberately deferred (brief asks for at least one to be explained at the defence):

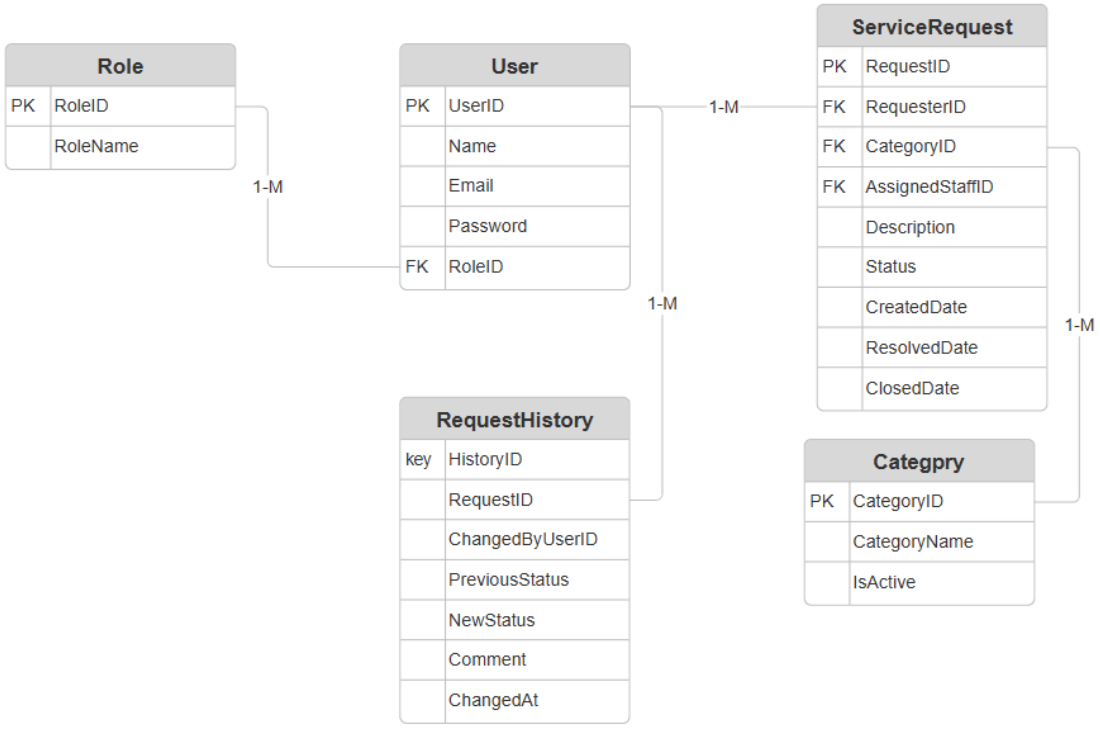
Deferred decision	**Why deferred**	**Evidence needed before deciding**
Horizontal scaling / separating Reporting into its own deployable	No user-volume or load data; complexity unjustified now	Measured response times against ASR-5 under expected load
Availability target and redundancy	M1 defines no availability requirement	Stakeholder confirmation of acceptable downtime
Notification channels beyond in-app (email/SMS)	Out of M1 scope, cost decision pending	Cost and licensing evidence; approved change request

---

Data and persistence baseline:

The purpose of this section is to convert approved M1 requirements into a structured format for managing information related to service requests. It centers on the core service-request lifecycle, covering aspects such as submission, categorization, assignment, status tracking, history, feedback, accountability, and management reporting. This document will serve as the foundational baseline, with potential for further attributes and constraints to be integrated during implementation as new evidence or requirements emerge.

Initial data model:



Entity relationship:

Relationship	cardinality	Reason
Role to User	1:M	One role can apply to multiple users.
User to ServiceRequest(requester)	1:M	A requester can submit multiple service requests
Category to ServiceRequest	1:M	A category can contain multiple service requests
User to ServiceRequest(assigned staff member)	1:M	A staff member can be assigned multiple service requests
ServiceRequest to RequestHistory	1:M	A request can have multiple history entries
User to ServiceRequest	1:M	A user can make multiple recorded changes

Ownership:

Entity	Lifecycle
Role	Controlled reference data which will be managed by authorized administration
User	Represents an account along with its assigned role
Service Request	Functions as a central business entity with a lifecycle from submission to resolution.
Request History	Dependent on service requests and tracks changes throughout its lifecycle.
Category	Controlled reference data which will be managed by authorized administration

Persistence decisions

Requirements driving the persistence decision:

The M1 baseline is the source of the following requirements

- Organized connections between users, requests, history, and categories
- Distinct identifiers for service requests
- Regulated categories
- Lifecycle and status tracking
- Auditable history
- Role-based access
- Dependable storage
- Management reporting
- Request filtering by category/status
- Atomic creation and modification of request data
- Protection of sensitive data
- Potential expansion

The team considered the following alternatives:

Relational database(such as PostgreSQL, MySQL, SQL server):

The advantages are as follows:

- Structured schema
- Foreign-key relationships
- Referential integrity
- Constraints
- Transactions
- Appropriate for request/history relationships
- Appropriate for managing reports and queries

Document database(MongoDB):

The advantages are as follows:

- Adaptable document structure
- Simple document shape modification
- Appropriate for applications where data is inherently document-oriented

Recommended decision:

The recommended persistence method for CivicConnect is relational persistence. The organized relationships and integrity constraints mentioned in M1 are a perfect fit for a PostgreSQL relational database.

PostgreSQL transactions are made up of multiple SQL statements which are combined into a single, atomic unit of work. This process protects the database from incomplete data changes and corruption by guaranteeing that either all operations are fully successful or none of them take effect (PostgreSQL, 2026). Within PostgreSQL foreign-key constraints connect two tables and maintain referential integrity. It guarantees that a value in a child table's column corresponds to a value in a parent table's main key or unique column. PostgreSQL offe3rs two core methods for backing up data, namely, logical and physical backups. Physical backups duplicate the actual data files that PostgreSQL uses, whereas logical backups generate SQL statements that can replicate your database's structure and data (Pawale, 2026).

Overall, PostgreSQL was chosen over the other relational database systems because it satisfies CivicConnect's anticipated data and reporting requirements while offering the necessary relational features, robust referential integrity, transaction support, and dependable backup capabilities. It also proved to be the more sensible option for the project's development and deployment requirements because it is open-source and does not require licensing.

Data integrity and transaction decision:

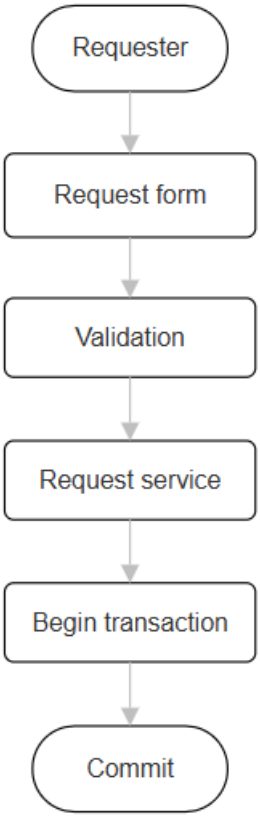
Upon receiving a service request the system must generate the following:

- The service request
- Its original status
- The creation date
- Relevant metadata
- And its initial requestHistory record.

The appropriate manner to handle these activities it to treat them as a single business activity. CivicConnect may have an unfinished audit trail if the request is created successfully but its original history is not documented.

The validation responsibilities for the system must be separate. The UI must be utilized for basic input validation, the service/business layer for handling business rules and transactions, whilst the database manages structural constraints and integrity of data.

The proposed flow should be as follows:



Technology stack:

For CivicConnect, the chosen technological stack is made up of C# with ASP.NET Core for the backend and application runtime, React for the frontend and PostgreSQL for persistence. The Npgsql PostgreSQL supplier will supply the application-to-database data access layer for Entity Framework Core. Automated unit testing will be done with xUnit, and build and test automation will be provided by GitHub actions. The M1 requirements for role-based access, service request management, status/history monitoring, auditability, reporting, data integrity, and maintainability are all supported by this stack.

Area	Selected technology	Reason
Frontend	React framework	CivicConnect requires several interactive views for requesters, staff, management, and administrators, including service-request forms, status updates, filtering, request history, and management reports. React was chosen because these interfaces can be created as reusable components thanks to React's component based architecture, which promotes consistency and maintainability throughout the application.
Backend	ASP.NET core MVC/C#	This decision offers an organized application architecture that can be used to divide persistence, business logic, and presentation. Additionally, CivicConnect's requester, staff, management and admin roles are directly supported by ASP.NET Core's built-in authentication and authorization features.
Persistence	PostgreSQL	The structured relationships between users, roles, categories, service requests, and request history in CivicConnectare best served by a relational database. Data integrity may be maintained via PostegreSQL's constraints and transaction features.
Data access	Entity framework core + Npgsql	Reduces the amount of database-access code that needs to be written by hand while offering PostegreSQL a data-access layer that is compatible with .Net.
Testing	xUnit	Offers a simple framework for automated business logic and service unit testing.
Build management	.NET SDK+ NuGet	This choice gives the chosen C#/.NET stack a standard build and dependency-management procedure. Through the project settings, dependencies can be reliably restored and version pinned.
CI	GitHub actions	It can automatically build and test pull requests, giving visual verification results prior to merging, and it works with the team's current GitHub workflow.
Deployment	ASP.NET Core + PostgreSQL	Instead of relying on a proprietary database or specialized runtime, the chosen technologies can be deployed as a conventional ASP.NET Core web application with a PostgreSQL database.

Comparison project constraints:

CivicConnect's functional requirements, team competency, timetable, and budget constraints were taken into consideration when choosing the technology stack. The interactive role-specific interfaces needed for M1, such as request submission, status viewing, filtering, and management dashboards, are supported by React. A structured backend for business logic, authentication, and role-based access control can be implemented using C# and ASP.NET Core. PostgreSQL was chosen because CivicConnect has data-integrity standards and defined relationships, and its open-

source licensing eliminatwes the need for additional database licensing fees. Development and dependency risk are also decreased by using well-known tools like Entity Framework Core, Npgsql, npm and NuGet. Lastly, the technologies may be connected with GitHub actiosnand the team's current GitHub process, making the stack feasible to develop within the project's timeline.

Version, compatibility and security assumptions:

The project will employ PostgreSQL 18 for persistence, React 19 for the front end, and.NET 10 or ASP.NET Core 10 for the back end. While Entity Framework Core with Npgsql will enable compatibility between the.NET application and PostgreSQL, frontend and backend dependencies will be managed by npm and NuGet, respectively, with versions documented in the project files to guarantee consistent development environments. The frontend and backend will communicate via HTTP-based APIs.

Throughout development, compatible stable versions and security updates will be implemented. While input validation, parameterized database access, and PostgreSQL constraints will be utilized to lower common data and injection threats, ASP.NET Core will manage authentication and role-based authorization. Additionally, the group will steer clear of superfluous dependencies and check dependency updates for security and compatibility issues before adoption.

Decision and risk management

An Architecture Decision Record (ADR) will document the chosen technology stack and the alternatives taken into consideration, along with the rationale behind each technology's selection and the pertinent compatibility assumptions. Risks including team familiarity, dependency compatibility, security flaws, and deployment concerns will be addressed by updating the project risk registry. During development, any major modifications to the technology stack will be compared to the initial specifications and recorded in a revised ADR.

Technology Architectural Design Record's:

ADR-001: React frontend

CivicConnect necessitates a flexible web interface that supports various roles and interactive features, such as staff assignment, management reporting, status viewing, request history, and service request submission. Therefore, maintainability and reusable user interface components must be supported by the frontend technology while staying within the team's development timeline and skill development.

Frameworks considered:

- Blazor
- Plain Html/CSS/Javascript

- Angular
- React
- Other frameworks used for frontend

Decision:

React was chosen because interactive role-specific interfaces and reusable UI components are supported by its component-based architecture. It offers CivicConnect enough capability without necessitating the intricacy of a more elaborate application architecture.

Positive outcomes:

- Reusable components
- Ideal for dashboards and interactive forms
- Extensive documentation and ecosystem
- Keeps backend AP and frontend duties separate.

Negative outcomes:

- Adds more dependencies than just basic Javascript
- Calls for team to learn more about npm and react.

Risk:

Maintaining compatible React and npm dependencies is essential, as is checking major dependency updates for compatibility or security flaws.

ADR-002: C#/ASP.NET Core backend

For the administration of service requests, status changes, authentication, role-based access control, validation, and database interaction, CivicConnect needs backend business logic. In addition to being appropriate for the team's skills and project timeline, the backend must provide a maintainable architecture.

Frameworks considered:

- C#/ASP.NET Core
- Node.js / express
- Other server-side frameworks

Decision:

Because it offers an organized framework for executing the application's business logic, authentication, and role-based authorization, C# with ASP.NET Core was chosen. Additionally, it has direct integration with PostgreSQL and Entity Framework Core.

Positive outcomes:

- Durable business logic framework
- Supports role-based access control and authentication
- PostgreSQL with Entity Framework core integration

Negative outcomes:

- Requires familiarity of the .Net ecosystem
- Greater structure within the framework than a basic Node.js/Express program

Risk;

The project is contingent on the chosen versions of Entity Framework Core, Npgsql, and.NET remaining compatible during development. Dependency versions will be controlled, and major updates will be reviewed before adoption because updates to these dependencies may cause compatibility problems or necessitate code changes.

ADR-003: PostgreSQL Relational Persistence

Using M1 data-integrity needs and structural relationships between entities like Users, Roles, Categories, Service Requests, and Request History were identified. Reliable reporting, transactions, and referential integrity must all be supported by the persistence system.

Databases considered:

- PostgreSQL
- MySQL
- MongoDB

Decision:

CivicConnect's structured relationships make PostgreSQL an excellent choice for relational persistence. In order to preserve consistent data, PostgreSQL offers transactions, foreign-key restrictions, and other database constraints. Additionally, it saves additional database licensing fees because to its open-source licensing.

Positive outcomes:

- Strong integrity between relationships
- Supports transactions
- Ideal for structured queries and reporting
- No extra fee for database licensing
- EF core compatibility via Npgsql

Negative outcomes:

- Requires well defined schema for relationships
- Migrations must be used to handle schema changes

Risk:

The chosen PostgreSQL version must continue to work with the database schema and migrations. Entity Framework Core and the Npgsql provider will be used for database access.

Updated RTM

Requirement	Persistence impact	Design	Technology	Implementation evidence	Verification
FR-001	ServiceRequest + initial RequestHistory	Request creation service	React + ASP.NET Core + PostgreSQL	Endpoint + models	Planned
FR-002	Category FK	Controlled category entity	ASP.NET Core + PostgreSQL	Category model and schema	Planned
FR-003	RequestID PK	Unique request identifier	PostgreSQL + EF core	Schema/model	Planned
FR-004	ServiceRequest.Status	Request retrieval	React + ASP.NET Core + PostgreSQL	Status endpoint	Planned
FR-005	RequestHistory	Request history relationship	React + ASP.NET Core + PostgreSQL	History model	Planned
FR-006	RequestHistory	Status-change handling	ASP.NET Core + PostgreSQL	Status service	Planned
FR-010	AssignedStaffID	Assignment responsibility	React + ASP.NET Core + PostgreSQL	Assignment endpoint	Planned
FR-011	Status + RequestHistory	Controlled status transition	ASP.NET Core + PostgreSQL	Status service	Planned
FR-012	RequestHistory	Action/comment recording	React + ASP.NET Core + PostgreSQL	History model	Planned
FR-014	RequestHistory	Management activity query	React + ASP.NET Core + PostgreSQL	Management query	Planned
FR-015	Status + dates	Reporting queries	React + ASP.NET Core authorisation+ PostgreSQL	Reporting API	Planned
NFR-019	User/Role + access rules	RBAC architecture	ASP.NET Core + PostgreSQL	Role implementation	Planned
NFR-020	RequestHistory/audit data	Audit trail	ASP.NET Core + PostgreSQL	History implementation	Planned
NFR-022	Persistent ServiceRequest	Transaction persistence	PostgreSQL + EF core	Migration + transaction logic	Planned
NFR-024	Database/application response	Performance-aware architecture	React + ASP.NET Core + PostgreSQL	Query/API implementation	Planned

A. Design Problems and Pattern Decisions

The problems discussed come from M1 baseline requirements for CivicConnect. The record format follows Nygard (2011). The problem is stated before pattern is named as well as project specific decision is record with is benefit and cost. The previous ADR is used as evidence

A.1 Design Problem 1 — Controlled request status lifecycle (DP-1)

Context: M1 2.1 defines a transition, for giving requesters honest visibility. FR-011 requires only verified transitions to be recorded while FR-013 lets authorised staff resolve or close. NFR-018 restricts actions by role.

Problem: These rules that determine legal or permissible transitions are used in multiple places, example: staff updates, resolve/close., if they are written as scattered conditions, one code path will eventually allow an illegal move. This could cause a corruption in the history that FR-014, FR-015 and NFR-020 depend on. Thus, we need to design the system to keep the rules in one place, which can be tested thoroughly

Lifecycle rules: Only 7 of the 36 transitions are legal in this case. Closed or rejected are terminal. The right-hand column records the policy that the rule must implement. While the authorisation, is decided outside of the lifecycle

From	To	Required input	Authorisation policy (Identity & Access)	Reqs
Submitted	Acknowledged	—	Authorised staff	FR-011, FR-006
Submitted	Rejected	Reason	Authorised staff	FR-011, FR-006
Acknowledged	In Progress	Assignee set	Assigned staff member	FR-010, FR-011
Acknowledged	Rejected	Reason	Authorised staff	FR-011, FR-006
In Progress	Resolved	Resolution note	Assigned staff member	FR-012, FR-013
Resolved	Closed	—	Authorised staff	FR-013
Resolved	In Progress (reopen)	Reason	Authorised staff	FR-011

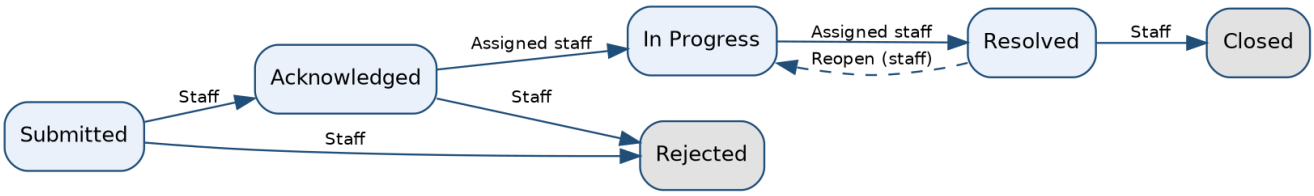


Figure A1 — Legal status transitions (solid = forward path, dashed = reopen). Matches PED §4.3.

ADR-006 — Use the State pattern for the request lifecycle

Field	Record
Status / date / owner	Proposed | 30/9/2026 | Person 3 (Accepted when the M2 baseline is signed)
Decision	Model each status as a state object behind a `RequestState` interface. `ServiceRequest.transitionTo(target, actor, note)` first asks Identity & Access whether the actor may act (ADR-003), then delegates to its current state, which either accepts the target and supplies the new state or rejects it with a reason. The change and its audit record are then committed together (ADR-004, ADR-005). States decide legality and required inputs; they never inspect roles.
Alternatives considered	(a) Conditional logic (if/else or switch) in the request service: simplest to write, but this is the form of code the State pattern is designed to replace when behaviour depends on state (Gamma et al., 1994, pp. 305–313; Freeman and Robson, 2020). (b) Data-driven transition table (map of from/to/required input): also centralised and lighter than State, and the closest alternative. (c) State pattern (chosen).
Why (c) over (b)	Transitions carry behaviour as well as permission: terminal states reject everything, In Progress requires an assignee, and Rejected/Resolved/Reopen require a note. Per-state classes keep that behaviour cohesive and testable one state at a time. Team judgement, to be defended at the presentation: a table would need extra condition code for the same cases.
A2 evidence	The pattern is documented by PED/ADR observer-based reactions are displayed in application code. Separate validation procedures are displayed by PED/ADR and application code. PED/ADR, data model and application transaction/validation logic. PED/ADR, architecture diagram and application interface implementation. Repository settings, PR history, CI workflow/results and project documentation.
Benefit	One owner for lifecycle rules (NFR-023, ASR-4); adding a status touches one class; each rule is unit-testable without a database (ASR-2 requires invalid transitions to be rejected 100% of the time in tests).
Complexity / trade-off	Gamma et al. (1994, pp. 305–313) note that State spreads behaviour over more classes than one conditional would and is worthwhile mainly when there are many states. Six states is near the lower limit, so this cost is real. Mitigation: states are stateless and shared, no framework is added, and `ServiceRequest` depends only on the interface, so falling back to option (b) is a contained change.
Affected components	Request Management: `ServiceRequest`, `RequestState` and concrete states, `RequestService`; the transitions endpoint (B.2); `StatusHistory` persistence; reporting queries for open/overdue.
If reversed	RTM rows FR-010, FR-011, FR-013 (design, implementation, verification), the transition test suite, Figures A1/A2, R10 and README module notes would change. The API contract in B.2 would not.
Traceability	FR-010, FR-011, FR-013, NFR-018, NFR-020, NFR-023; ASR-2, ASR-4; ADR-003, ADR-004; risks R6, R10.

A.2 Design Problem 2 — Audit trail and requester feedback on every status change (DP-2)

Context: NFR-020 require major changes to be tracked. It requires a status, and responsible staff member to be added. FR-006 only needs feedback from the request when the status get changed. M1 3.3 we decided to defer SMS/emails, so in-app pop ups and notifications are within scope but other channels are expected later on for ASR-4. ADR-004 is a fix for the rule that allows audit record to be written with the same transaction as the change. It prevents the end user to be connected to each part of every lifecycle.

Problem: When successful change is completed a audit record and requester-visible update must be produced. This applies to every code path that changes a request. This is to provide a robust change log and support debugging. But if a Request Management calls audit and notification code directly, the core lifecycle is coupled to both processes. A new change means editing core present/future code will forget a call silently and breaks auditability. The audit record must never be missing when a change is committed.

ADR-005 — Use the Observer pattern (synchronous, in-process) for status-change side effects

Field	Record
Status / date / owner	Proposed | 30/9/2026 | Person 3 (this is the ADR-005 candidate Person 1 asked the team to assign)
Decision	On a successful change, Request Management publishes a `StatusChanged` (or `AssignmentChanged`) event through `RequestEventPublisher`, which notifies subscribed `RequestEventListeners` (Gamma et al., 1994, pp. 293–303). Two listeners are registered in one composition point: `AuditTrailListener` (Audit module; appends a `StatusHistory` row) and `NotificationListener` (Notification module; records an in-app update through a `NotificationChannel`). Both run synchronously inside the same unit of work as the change (Fowler, 2002), so the transaction either commits all of it or none (Kleppmann, 2017, ch. 7). This is the mechanism that realises ADR-004; it is not the asynchronous "separate process or event" that ADR-004 rejected.
Dependency direction	The event types and `RequestEventListener` interface are part of Request Management's published interface. Audit and Notification implement that interface and never call Request Management back, in line with Person 1's rule that dependencies point inward (Martin, 2017) and rule 3 in PED §4.1. Confirm rule 3 is intended to permit implementing a published listener interface; if not, move the event contract into a small shared kernel and update ADR-002.
Extension point (ASR-4)	The Notification module exposes a `NotificationChannel` interface. An in-app channel is implemented now; email/SMS (M1 §3.3, deferred) become new channel classes attached after commit, so an external outage cannot block staff work.
Alternatives considered	(a) Direct calls from Request Management to audit and notification code: least indirection, but coupling and the forgotten-call risk above. (b) Asynchronous queue or message broker (Hohpe and Woolf, 2003): decouples fully, but adds infrastructure and eventual consistency, which works against "exactly one audit record per change" (Newman, 2021) and is disproportionate for a three-person team on a free-tier assumption (M1 §4). (c) Database triggers for audit: rejected in ADR-004 (logic hidden from code review, harder to test); also cannot produce requester updates. (d) Observer, in-process and synchronous (chosen).
A2 evidence	The pattern is documented by PED/ADR observer-based reactions are displayed in application code. Separate validation procedures are displayed by PED/ADR and application code. PED/ADR, data model and application transaction/validation logic. PED/ADR, architecture diagram and application interface implementation. Repository settings, PR history, CI workflow/results and project documentation.
Audit content	Each history row records who, what and when: actor, request, from-status, to-status, timestamp and note, following the event attributes recommended for security logging (OWASP Foundation, n.d.a). Rows are append-only (ADR-004).
Benefit	Lifecycle code no longer knows about auditing or notification (ASR-4, NFR-023); a new channel is a new class; the audit guarantee is enforced in one place; listeners are testable in isolation.
Complexity / trade-off	Gamma et al. (1994, pp. 293–303) note that observers add indirection and can produce unexpected cascading updates, so control flow is harder to follow. Listener order and failure behaviour must be documented. Synchronous listeners add latency to each change; the NFR-024 target (95% of regular operations within 3 seconds, as clarified by CR-001) is expected to hold at this scale and will be measured in M3. Failure rule: if the audit write fails, the change is rolled back, deliberately favouring accountability (M1 §1.1). Mitigation: only two listeners and a test that fails if the audit listener is not registered.
Affected components	Request Management (`RequestService`, publisher, listener interface), Audit (`AuditTrailListener`, `StatusHistory`), Notification (`NotificationListener`, `NotificationChannel`), history and notifications endpoints (B.2).
If reversed (e.g. to a queue)	The transaction rule, the deployment footprint (extra service), R11, README run instructions, Person 2's persistence ADR and NFR-022/NFR-024 verification would change; so would ADR-004 and RTM rows FR-006, FR-014, NFR-020.

Field	Record
Traceability	FR-006, FR-012, FR-014, NFR-020, NFR-022, NFR-023; ASR-2, ASR-4; ADR-002, ADR-004; risk R11.

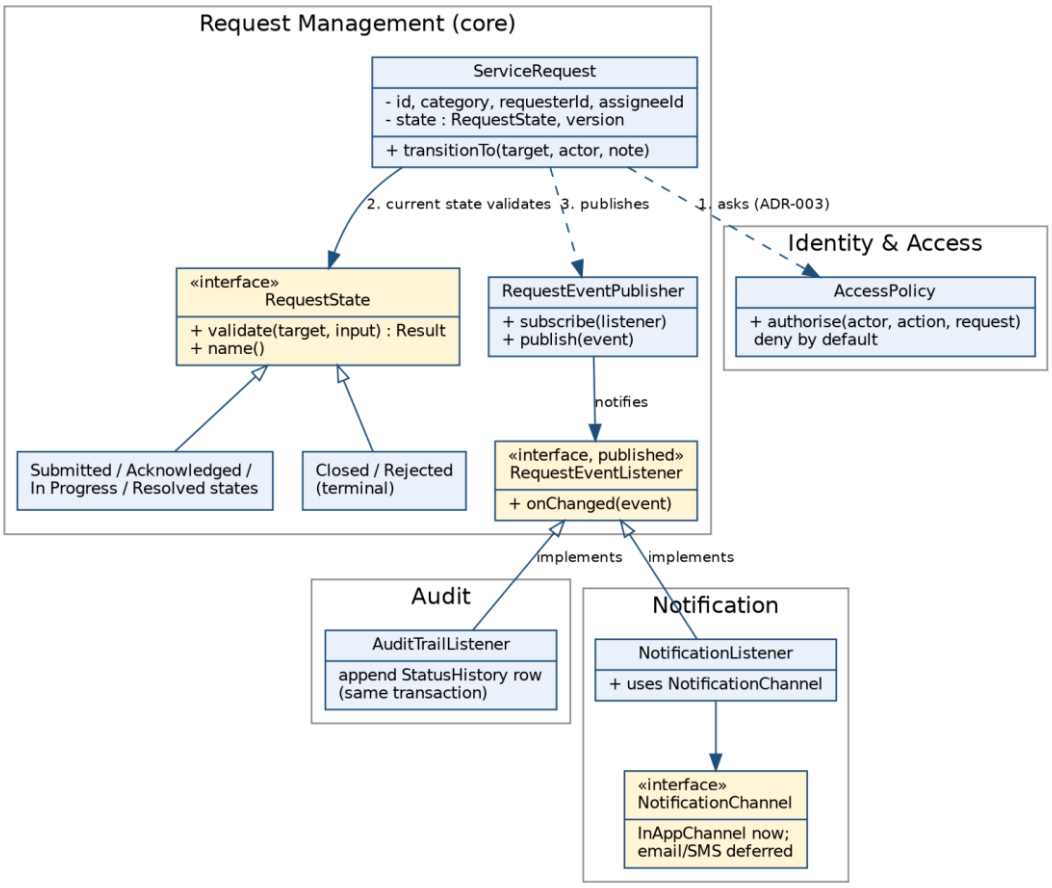


Figure A2 — Design diagram: State (DP-1) and Observer (DP-2) across the Request Management, Identity & Access, Audit and Notification modules.

A.3 How the two decisions work together

A transition request runs in this order: authenticate, authorise through Identity & Access (ADR-003), validate through the current state (ADR-006), apply the change and publish the event inside one transaction (ADR-004, ADR-005). If authorisation or the state rejects the request, nothing is published, so the history contains only verified transitions, which is the acceptance criterion of FR-011 and the ASR-2 target.

Initial application evidence: Lifecycle evidence lives under the “feature/status-lifecycle” branch on the repo.

B. Interface / API Decisions

B.1 Boundaries and mechanism

CivicConnect has one genuine network boundary: the browser client talking to the application server (PED §4.4). The six modules are logical units inside one deployable, so calls between them are ordinary in-process calls through published interfaces (ADR-002). No microservice or external integration is created, consistent with the M1 exclusion of third-party/municipal integrations and the brief's proportionality rule.

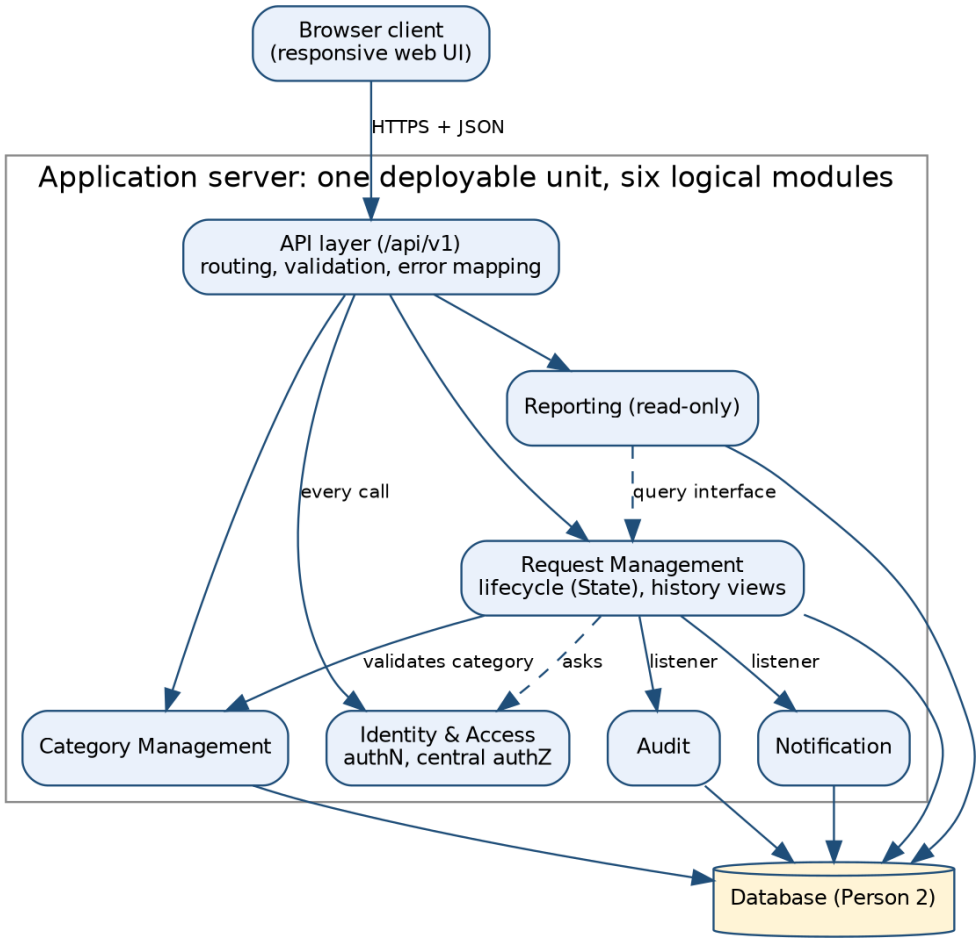


Figure B1 — Modules and the single network boundary (browser ↔ API layer).

ADR-007 — Resource-oriented JSON API over HTTPS, versioned under /api/v1

Field	Record
Status / date / owner	Proposed | 30/9/2026 | Person 3
Decision	A resource-oriented API in the REST style (Fielding, 2000, ch. 5) using standard HTTP methods and status codes (Fielding, Nottingham and Reschke, 2022). The major version is in the URL path (`/api/v1`), the approach used in Google's published versioning guidance (Google, n.d.). Actions that are not simple updates (assignment, status transition) are sub-resources so lifecycle and access rules are always applied server-side. The contract is written as an OpenAPI description (OpenAPI Initiative, 2021) kept in the repository. Errors use the RFC 9457 problem-details format (Nottingham, Wilde and Dalal, 2023).

Field	Record
Alternatives considered	GraphQL (GraphQL Foundation, 2021): flexible queries, but extra tooling and a harder per-field authorisation surface for a small, fixed set of screens. RPC-style endpoints: simple, but a poorer fit to requests as resources. Server-rendered form posts: valid if Person 2 selects a server-rendered stack; the service contract in B.2 stays the same and only the outermost layer changes.
A2 evidence	The pattern is documented by PED/ADR observer-based reactions are displayed in application code. Separate validation procedures are displayed by PED/ADR and application code. PED/ADR, data model and application transaction/validation logic. PED/ADR, architecture diagram and application interface implementation. Repository settings, PR history, CI workflow/results and project documentation.
Benefit	Predictable, easy to test and document; maps directly onto FR-001 to FR-016; authorisation and validation are enforced in one place (ASR-1); a machine-readable contract supports later contract tests.
Trade-off	Several small endpoints and a versioning obligation; some screens need more than one call, which is acceptable at this scale.
If reversed	B.2 endpoint table, the OpenAPI file, README API section, RTM interface column, client code, contract tests and R12.
Traceability	FR-001 to FR-016, NFR-017 to NFR-019, NFR-021; ASR-1, ASR-5; ADR-001, ADR-003.

B.2 Initial endpoint contract

Roles: Req = Requester, Stf = Staff, Mgt = Management/Admin. Authentication (NFR-017) applies to every endpoint; the mechanism is a technology decision ADR-001 to ADR-004. Every endpoint passes through Identity & Access, and every endpoint that takes an object identifier checks that the caller may access that object, the weakness ranked first in the OWASP API Security Top 10 (OWASP Foundation, 2023).

Method & path	Purpose / payload	Roles	Reqs	Success / key errors
POST /api/v1/requests	Create request: category id, description, details	Req	FR-001, 002, 003	201 + id; 400/422 missing field or unknown category
GET /api/v1/requests	List with filters: status, category, priority, text search, sort	Req (own only), Stf, Mgt (in scope)	FR-004, 005, 007, 008, 016	200; out-of-scope rows never returned
GET /api/v1/requests/{id}	Full request detail	Req (own), Stf, Mgt	FR-004, 009	200; 403 / 404
POST /api/v1/requests/{id}/assignment	Accept or assign owner	Stf (authorised)	FR-010	200; 403; 409 already assigned
POST /api/v1/requests/{id}/transitions	Body: target status, note, expected version	Stf (policy in A.1)	FR-011, 013	200; 409 illegal transition or stale version; 403; 422 note missing
POST /api/v1/requests/{id}/comments	Add action / resolution note; internal flag	Stf	FR-012	201; 403
GET /api/v1/requests/{id}/history	Status and ownership history	Req (own, non-internal entries), Stf, Mgt	FR-005, 014, NFR-020	200; internal notes hidden from requester (NFR-019)
GET /api/v1/notifications	In-app status updates for the caller	Req	FR-006	200

Method & path	Purpose / payload	Roles	Reqs	Success / key errors
GET /api/v1/categories	Controlled category list	All authenticated	FR-002	200
POST/PUT /api/v1/categories	Maintain category list	Admin	FR-002	201/200; 403
GET /api/v1/reports/requests-summary	Open / overdue / resolved / closed counts by category and status	Mgt	FR-014, 015, 016	200; 403 for non-management. “Overdue” depends on CR-002

B.3 Cross-cutting interface rules

Concern	Decision (level currently known)
Validation	Server-side for every write: required fields, category must exist in the controlled list, note required for Reject/Resolve/Reopen, lengths bounded. Client checks are convenience only (NFR-021).
Error / failure behaviour	Errors are returned as `application/problem+json` (Nottingham, Wilde and Dalal, 2023) with the standard members (`type`, `title`, `status`, `detail`, `instance`) plus two extension members: `errors[]` for field-level validation messages and `correlationId` for support. Status codes follow HTTP semantics (Fielding, Nottingham and Reschke, 2022): 400 malformed, 401 unauthenticated, 403 not permitted, 404 not found or deliberately not disclosed (the RFC allows 404 in place of 403 to hide existence), 409 conflict (illegal transition, stale version), 422 semantic validation failure, 500 with no internal detail. A rejected transition returns the reason supplied by the state object.
Security boundary	Authentication on every route; role and ownership checks made by Identity & Access (ADR-003), deny by default, so a direct API call cannot bypass RBAC. Object-level and function-level checks are both required (API1 and API5 in OWASP Foundation, 2023). Requester queries are filtered to own records; internal comments are stripped from requester responses (NFR-019). No secrets in the repository (D.4).
Concurrency	Transition and assignment calls carry an expected version. A stale value returns 409, so two staff cannot silently overwrite each other; this is the optimistic offline lock described by Fowler (2002).
Change / version	Additive changes (new optional field, new endpoint) stay in v1; removing, renaming or retyping a field is breaking and requires a new major version plus an ADR/change record (Google, n.d.). The OpenAPI description lives at `docs/api/openapi.yaml`: Status deferred for future implementation.

Deferred (recorded, not invented): pagination limits, rate limiting, external notification channels and any integration with outside systems. See H.3.

C. Application Documentation and Repository Structure

The README is delivered as a separate repository file (`README.md`) prepared with this document. It must match the code and this PED. Configuration guidance in it follows the twelve-factor practice of keeping configuration in the environment (Wiggins, 2017).

Required item (M2 brief §8)	Where covered	Status
Purpose, current implementation status	README §1–2	Drafted; update status each merge
Prerequisites, runtime and dependency versions	README §3	Fill from Person 2 stack ADR and authoritative sources
Setup and run instructions	README §4	Drafted with placeholders for stack commands
Repository structure mapped to modules	README §5 and table below	Drafted; adjust to real folders

Required item (M2 brief §8)	Where covered	Status
Database / schema / migrations	README §6	Pending Person 2 ERD and migration tool
Interfaces / API endpoints	README §7, `docs/api/openapi.yaml`	Drafted from B.2
Environment / configuration without secrets	README §8, `.env.example`	Drafted
Module responsibilities and design-pattern application	README §9	Drafted from A and B
Testing, limitations, TODOs	README §10–11	Drafted; must reflect real tests
Links to PED / RTM / ADRs	README §12	Add links once PED v2.0 is in the repository

C.1 Repository structure aligned to the architecture (Person 1 modules)

Module (PED §4.1)	Proposed location	Published interface / notes
Identity & Access	`identity_access/`	`AccessPolicy.authorise(actor, action, resource)`; deny by default
Request Management (core)	`request_management/`	Lifecycle (`RequestState` classes), `RequestService`, `RequestEventPublisher` and `RequestEventListener`
Category Management	`category_management/`	Category lookup and validation interface
Audit	`audit/`	`AuditTrailListener`; append-only `StatusHistory`; no update/delete path
Notification	`notification/`	`NotificationListener`, `NotificationChannel` (in-app now)
Reporting	`reporting/`	Read-only query interface over Request Management data
API layer, client, docs, CI	`api/`, `client/`, `docs/`, `.github/`	API layer contains no business or authorisation rules

Folder names mirror the modules so that dependency violations are visible in code review (ADR-002).

Code documentation rule (team). Comment intent, not mechanics. Every state class states which transitions it permits and why; every listener states its failure behaviour; each endpoint handler and test names the requirement it serves (for example `// FR-011`). Tagging code with requirement IDs keeps traceability followable in both directions, from requirement to code and from code back to requirement (Gotel and Finkelstein, 1994).

D. GitHub Governance & Repository Evidence

M1 governance is retained unchanged: protected `main`, no direct pushes, pull requests required, two approvals, stale approvals dismissed, most recent push approved by someone other than the pusher, linear history, force pushes blocked and deletions restricted. GitHub documents these as branch protection rules that enforce required

reviews and passing status checks before merge (GitHub Docs, n.d.a). Repository: https://github.com/Armand-JV/SEN381-Project. Members: Armand Vermeulen (Armand-JV), Riegardt Weich (xStoffer), Ditheto Phaho (Deee-dot).

D.1 M2 working conventions

Area	Convention
Issues	One issue per task, labelled by area (`docs`, `design`, `api`, `data`, `governance`, `test`), linked to the RTM requirement ID, all in the M2 milestone.
Branches	As per M1: `feature/<name-or-id>`, `bugfix/<issue-id>`; add `docs/<name>` for PED, README and RTM changes and `chore/<name>` for tooling.
Commits	Short imperative subject with the requirement or issue reference, e.g. `Add ResolvedState transition rules (FR-013, #14)` (imperative subject lines: Beams, 2014). Commit as work is done; no bulk uploads.
Pull requests	Template asks: what changed, linked issue, requirement/ADR IDs, how it was tested, whether AI assisted and how it was verified, and whether docs were updated. Documentation and RTM changes are reviewed like code.
Reviews	Every substantive PR needs both other team members (two independent approvals). Empirical work on code review at Microsoft found that, besides finding defects, review spreads knowledge and team awareness (Bacchelli and Bird, 2013), which is why all three members review. Turnaround target: 48 hours. Reviewers also enforce Person 1's "no cross-module internals" checklist item (ADR-002).
Changes to baselined decisions	A new ADR supersedes the old one; the old ADR stays, marked Superseded with the reason. No silent rewrites.

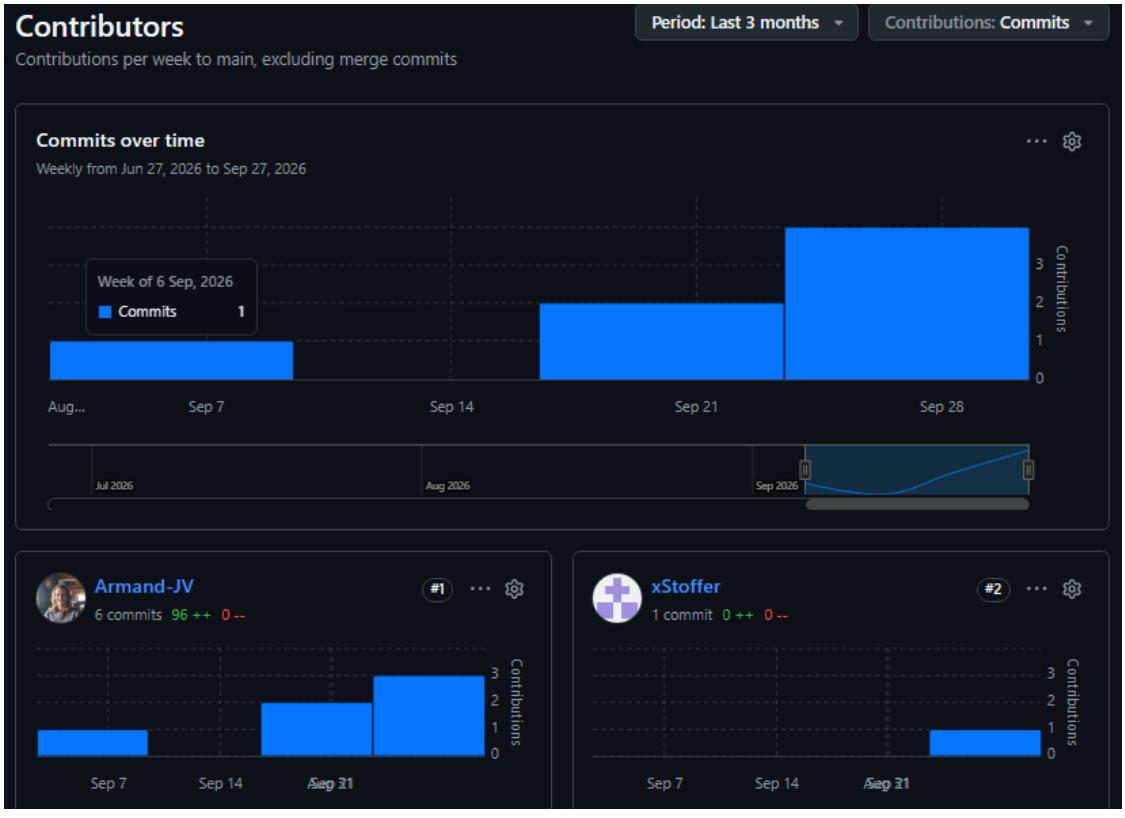
ADR-008 — Adopt minimal automated checks now; defer a mature pipeline

Field	Record
Status / date / owner	Proposed | 30/9/2026 | Person 3
Decision	Adopt now: a GitHub Actions workflow (GitHub Docs, n.d.c) that builds the project and runs the automated tests on every pull request, set as a required status check (already the intent of M1); `.gitignore`, `.env.example`, PR and issue templates. Merging small changes frequently onto a tested mainline is the core of continuous integration (Fowler, 2006).
Secret and dependency protection	Enable secret scanning with push protection and dependency alerts where the repository supports them. GitHub documents secret scanning as available for public repositories, and for organisation-owned repositories only with an additional paid product (GitHub Docs, n.d.b). GitHub visibility set to public with only contributors access to change repo.
Deferred	Coverage thresholds, deployment automation, container builds, static-analysis gates and release tagging: these are the later stages of a deployment pipeline (Humble and Farley, 2010). Evidence needed: Person 2's deployment decision and a stable test suite.
A2 evidence	Use pull requests, protected main, feature branches, 2 approvals, and CI quality gates.
Benefit / trade-off	Fast, free feedback that protects `main`. Cost: workflow maintenance, and a flaky build blocks two-approval merges (R13).
If reversed	README testing section, PR template, R13 and the RTM verification column.

D.2 Evidence checklist (M1 items carried forward, M2 items added)

Evidence	Status	Where to point the assessor
Repository created; protected main (M1 rules)	Done	https://github.com/Armand-JV/SEN381-Project/settings/rules/22544388
Issues and task board for M2	Open	https://github.com/Armand-JV/SEN381-Project/issues
Feature branches for design, API, docs and code	Open	https://github.com/Armand-JV/SEN381-Project/branches
Pull requests with two independent approvals	Done	https://github.com/Armand-JV/SEN381-Project/pull/2
Review of documentation PRs (PED, RTM, README)	Done	https://github.com/Armand-JV/SEN381-Project/pull/1
Progressive commit history by all members	Done	https://github.com/Armand-JV/SEN381-Project/graphs/contributors?from=6%2F27%2F2026
CI workflow running on PRs	Done	https://github.com/Armand-JV/SEN381-Project/actions/workflows/test.yml
No secrets committed; `.env` not present	Done	https://github.com/Armand-JV/SEN381-Project
README and technical docs present and current	Drafted (Section C)	https://github.com/Armand-JV/SEN381-Project/blob/main/README.md

D.3 Contribution record



D.4 Secrets and configuration

Configuration is read from environment variables at start-up; configuration that varies between deployments belongs in the environment, not in code (Wiggins, 2017). `.env` is git-ignored and `.env.example` lists names with dummy values only. Secrets are never committed, following OWASP's secrets-management guidance (OWASP Foundation, n.d.b); a credential committed by mistake is treated as compromised and rotated, not merely deleted from history.

E. RTM v2.0 — Design/Interface, Implementation and Verification Columns

The RTM continues from M1 (same IDs and wording). Person 1's ASR link and architecture module columns and Person 2's data and technology columns merge in by requirement ID. The columns below are Person 3's. Status uses controlled values: Approved (baselined, not started), In Development, Implemented, Changed, Deferred. Implementation evidence names the planned artefact and is replaced by the real branch/PR/file once it exists; verification is Planned until a test exists. The aim is a matrix that can be followed from requirement to code and back (Gotel and Finkelstein, 1994). Planned tests follow the test-pyramid idea of many fast unit tests and fewer integration tests (Fowler, 2012).

ID	Design / interface decision	Implementation evidence	Verification evidence	Status	ADR / risk
FR-001	B.2 POST /requests; server-side validation	Create-request handler + validation	Planned: VT-001 missing field rejected; valid creates request	Approved	ADR-007; ASR-3
FR-002	B.2 GET /categories; category must exist	Category validation	Planned: VT-002 unknown/absent category rejected	Approved	ADR-007
FR-003	Identifier generated by persistence layer	Request id on create	Planned: VT-003 ids unique	Approved	ADR-007; ASR-3
FR-004	B.2 list/detail; ownership filter	Query with requester filter	Planned: VT-004 requester sees only own	Approved	ADR-003, 007; R6
FR-005	B.2 list and /history	History view	Planned: VT-005 previous requests listed	Approved	ADR-005, 007
FR-006	DP-2 Observer; notifications endpoint	NotificationListener, InAppChannel	Planned: VT-006 notification row per accepted transition	Approved	ADR-005; R11
FR-007	Central authorisation (ADR-003); B.2 list	Visibility policy	Planned: VT-007 out-of-scope request unavailable	Approved	ADR-003, 007; R6
FR-008	B.2 list filters and sort	Filter/sort parameters	Planned: VT-008 results match selected attribute	Approved	ADR-007; ASR-5
FR-009	B.2 GET /requests/{id}	Detail handler	Planned: VT-009 all attributes returned	Approved	ADR-003, 007

ID	Design / interface decision	Implementation evidence	Verification evidence	Status	ADR / risk
FR-010	B.2 assignment; owner recorded	Assignment handler; «PR»	Planned: VT-010 unauthorised change refused; assignee stored	Approved	ADR-003, 006, 007
FR-011	DP-1 State; B.2 transitions	RequestState + concrete states, transitions handler; «branch/PR»	Planned: VT-011 7 legal transitions accepted, 29 rejected; role cases	In Development «confirm»	ADR-004, 006; R10
FR-012	B.2 comments; internal flag	Comment handler; «PR»	Planned: VT-012 note stored and retrievable	Approved	ADR-005, 007
FR-013	DP-1 Resolved/Closed states	ResolvedState, ClosedState; «PR»	Planned: VT-013 resolve/close visible to requester	Approved	ADR-006
FR-014	DP-2 history; B.2 /history, /reports	AuditTrailListener + history query; «PR»	Planned: VT-014 ownership change visible to management	Approved	ADR-004, 005
FR-015	B.2 /reports/requests-summary	Summary query; «PR»	Planned: VT-015 counts correct (needs CR-002 overdue rule)	Approved	ADR-007; CR-002
FR-016	B.2 list and report filters	Filter parameters; «PR»	Planned: VT-016 filter by category/status	Approved	ADR-007
NFR-017	B.3 authentication on every route	Auth middleware; «PR»	Planned: VT-017 unauthenticated call refused	Approved	ADR-003, 007
NFR-018	Central authorisation; DP-1 order of checks	AccessPolicy; «PR»	Planned: VT-018 wrong role refused, right role allowed	Approved	ADR-003, 006; R6
NFR-019	B.3 internal notes stripped for requesters	Response filtering; «PR»	Planned: VT-019 requester cannot retrieve internal note	Approved	ADR-003, 007; R6
NFR-020	DP-2 mandatory audit listener in same transaction	AuditTrailListener, StatusHistory; «branch/PR»	Planned: VT-020 change writes history; failed audit rolls back	In Development «confirm»	ADR-004, 005; R11
NFR-021	B.3 validation and problem-details errors	Validation messages, confirmation response; «PR»	Planned: VT-021 invalid input gives clear message	Approved	ADR-007
NFR-022	DP-2 unit of work; persistence per Person 2	Persistence layer; «PR»	Planned: VT-022 stored request retrievable after restart	Approved	ADR-004, 005; R8
NFR-023	Modules per Figure B1; patterns per A	Repository structure mirrors modules; «PR»	Planned: review checklist on every PR	Approved	ADR-001, 002, 005, 006; R10
NFR-024	Synchronous listeners kept light	n/a until M3 load check	Planned (M3): timing test, 95% within 3 s (CR-001)	Approved	ADR-005; ASR-5; R9

E.1 End-to-end trace (required demonstration): FR-011

This is the path Person 1 suggested (PED §7), completed with Person 3's decisions. Highlighted items need real repository evidence.

Step	Evidence
Requirement	FR-011: authorised staff update requests through controlled, verified transitions (Staff; High).

Step	Evidence
	ASR-2 (trustworthy lifecycle and audit trail); ASR-1 for who may act; M1 Quality and Security constraints; risks R6, R10.
Architecture responsibility	Request Management (lifecycle) with Audit and Identity & Access (PED §4.1); API layer only maps HTTP to service calls (Figure B1).
Data decision	Append-only `StatusHistory` written in the same transaction (ADR-004) and a version field on the request for optimistic concurrency.
Design / interface decision	ADR-006 State pattern; ADR-005 Observer for audit; ADR-007 POST `/api/v1/requests/{id}/transitions`.
Technology / ADR	React Frontend, ASP.NET C# backend. PostgreSQL database. ADR-008 for the automated check that runs the tests.
Initial verification	VT-011: all 7 legal transitions accepted for an authorised actor; the other 29 ordered pairs rejected; requester and unassigned staff refused; exactly one history row per accepted transition and none for a rejected one.

F. Risk Register v2.0

Person 3 owns the register. M1 risks R1–R4 are preserved with an M2 review note; Person 1's architecture risks R5–R9 are merged unchanged; Person 3's new risks continue from R10. Person 2 adds data, technology, deployment and cost risks from R19. Priority uses the M1 scale (Critical / High / Medium / Low).

F.1 M1 risks — M2 review

ID	Risk	Priority	M2 review note	Status
R1	Stakeholder requirements change after baseline	Critical	Controlled through CR-001 to CR-003 and ADR supersession; any requirement change updates RTM and this register.	Open
R2	Poor Git collaboration causes conflicts	High	Conventions in D.1 tighten branch and commit practice; conflicts to date.	Open — monitored
R3	Security requirements overlooked	High	Addressed by ADR-003 and B.3 (central server-side authorisation); see R6. Verification still Planned, so remains open	Open
R4	Schedule delays from unequal workload	Medium	See R17; contribution table D.3 added as monitoring evidence	Open

F.2 Architecture risks contributed by Person 1 (merged unchanged)

ID	Risk	Prob.	Impact	Priority	Mitigation	Owner	Contingency	Status
R5	Module boundaries erode inside the monolith	Medium	Medium	Medium	ADR-002 rules, PR checklist item, folder structure mirrors modules	Architecture lead	Refactoring task before next milestone	Open

ID	Risk	Prob.	Impact	Priority	Mitigation	Owner	Contingency	Status
R6	Access control bypassed through an endpoint that skips the central check	Medium	High	High	ADR-003; authorisation tests per endpoint; review checklist	Team	Security review and fix before release; log as change	Open
R7	Architecture over-engineered relative to team capacity	Low	Medium	Low	Proportionality principle (ADR-001); revisit if a module has no independent reason to exist	Architecture lead	Merge modules via controlled ADR change	Open
R8	Single application instance and single database are single points of failure; no availability target	Medium	Medium	Medium	Persist-then-confirm (ASR-3); backup approach (Person 2); availability deferred	Team	Restore from backup; document recovery steps	Open
R9	Free-tier hosting limits conflict with ASR-5 or ASR-3	Medium	Medium	Medium	Record limits as dependency assumption (M1 Cost constraint); measure early	Person 2	Move to paid tier or simplify reporting queries	Open

F.3 New risks from design, interface, governance and implementation (Person 3)

ID	Risk	Prob.	Impact	Priority	Mitigation	Owner	Contingency	Status
R10	State and Observer patterns add complexity beyond what six states and two listeners justify (Gamma et al., 1994)	Medium	Medium	Medium	Stateless shared states; only two listeners; review against ADR-005/006; low-cost fallback to a transition table	Person 3	Supersede the ADR and replace state classes behind the same interface	Open
R11	Audit or notification write fails, causing lost history or blocked staff actions	Medium	High	High	Audit in same transaction (ADR-004/005); test that fails if the audit listener is unregistered; external channels only after commit	Person 3 with Person 2	Reconcile from request table; documented manual correction procedure	Open
R12	API contract drifts between UI and backend	Medium	Medium	Medium	OpenAPI description as the contract (OpenAPI Initiative, 2021); versioning rule; contract tests once UI exists	Person 3	Freeze v1 fields; change via ADR	Open

ID	Risk	Prob.	Impact	Priority	Mitigation	Owner	Contingency	Status
R13	Two-approval rule or a failing CI check stalls merges when a member is unavailable	Medium	Medium	Medium	48-hour review target; small PRs; reviewers rotate; keep the workflow simple	Team	Recorded temporary change to approvals via change control, then restored	Open
R14	Secrets or credentials committed to the repository (OWASP Foundation, n.d.b)	Low	High	High	.gitignore, .env.example, secret scanning if available, PR checklist	Person 3	Rotate credential immediately; document incident	Open
R15	AI-generated code, text or references merged without genuine verification	Medium	High	High	AI Register with verification evidence (G); PR template asks about AI use; each member explains their own code; sources opened and checked	Team	Rewrite or remove unverified material; note in register	Open
R16	README, PED, RTM and code diverge	Medium	Medium	Medium	Docs updated in the same PR as code; RTM reviewed each merge	Person 3	Documentation sync PR before assessment	Open
R17	Contribution evidence is uneven or bulk-committed near the deadline	Medium	High	High	Weekly review of D.3; issues assigned per member; commit as work is done	Team	Reassign tasks and record in change log (links to R4)	Open
R18	Later A2 evidence contradicts a baselined design decision	Low	Medium	Medium	ADRs cite A2 sources; decisions reversible behind interfaces	Team	Controlled change: new ADR supersedes, old kept with reason	Open

G. AI Usage Register v2.0

M1 entries are retained. New entries cover the AI assistance used for these Person 3 sections and for suggesting sources. Verification stays Pending until the named person has actually done the check; do not mark it complete before then.

Date	Artefact	AI use	Human verification
07 September	Risk Register	Table Structure	Confirmed project requirements match table structure (Edited)
07 September	Forward Engineering Considerations	Strictly Consideration column	Researched common considerations for similar projects
08 September	Engineering Decision Log	Decision confirmation by AI review	Decisions created by Engineer

Date	Artefact	AI use	Human verification
30 September	DP-1 / DP-2 problem statements; ADR-005, ADR-006	AI drafted problem framing, alternatives, ADR wording and diagram source	Pending: Person 3 to check alternatives against A2 sources, confirm the decisions are the team's own and be able to explain them
30 September	Interface / API decisions; ADR-007	AI drafted endpoint table, error format and rules	Pending: check against Person 1 architecture and Person 2 stack; confirm each endpoint traces to an FR
30 September	GitHub governance section D; ADR-008	AI drafted conventions and templates	Pending: confirm settings and plan features match the repository
30 September	README and application documentation	AI drafted structure and text	Pending: run setup on a clean machine; fix versions and commands against authoritative sources
30 September	RTM columns; Risk Register R10–R18 and merge of R5–R9	AI drafted rows, test identifiers, risks	Pending: replace planned evidence with real links; team review of probability, impact, ownership
30 September	In-text citations and Section I references (Person 3 sections)	AI suggested sources; existence and details of seven sources were checked online during drafting (see I.2)	Pending: each member opens the sources they cite, confirms the sentence is supported, checks page numbers, and enters access dates
30 September	Person 1's references (Section 10 of their draft)	AI-suggested sources flagged by Person 1 for checking	Pending: Person 1 to verify; Person 3 to record the outcome here

H. Version History, Decision Log and Open Items

H.1 Version history (extends the M1 and Person 1 tables)

Version	Date	Author	Changes
0.1	06 September	Armand	Work split and format (M1)
0.8	08 September	Team	Each member integrates sections (M1)
1.0	09 September	Team	Verified baseline for submission (M1)
1.1	09 September	Riegardt	M1 review findings logged as CR-001 to CR-003
1.8	30 September	Armand and Ditheto	Design problems, ADR-005 to ADR-008, interface decisions, RTM design/implementation/verification columns, risks R10–R18 and merge of R5–R9, AI Register, governance updates, citations.
2.0	30 September	Team	Architecture, Technology & Initial Design Baseline approved

H.2 Engineering Decision Log (extends the M1 table; Person 1 ADRs shown for completeness)

ID	Decision	Status	Reason
D1	Use GitHub controlled repository	Accepted (M1)	Source control and governance
D2	Tech stack	Accepted	Evidence for team experience and collaboration
D3	Requirement IDs follow FR/NFR numbering	Accepted (M1)	Traceability
ADR-001 to ADR-004	Modular monolith; module rules; central authorisation; lifecycle + audit in one transaction	Proposed (Person 1)	See PED §5
ADR-005	Observer (synchronous, in-process) for audit and notification	Proposed/	Decoupling with the ADR-004 audit guarantee (DP-2)
ADR-006	State pattern for request lifecycle	Proposed	Central, testable lifecycle rules (DP-1)
ADR-007	Resource-oriented JSON API under /api/v1, RFC 9457 errors	Proposed	One proportionate network boundary
ADR-008	Minimal CI checks now, mature pipeline deferred	Proposed	Progressive adoption per brief §10

H.3 Deliberately deferred decisions and evidence still needed

Deferred decision	Evidence required before deciding
External notification channels (email/SMS)	Cost and provider limits (Person 2); demonstrated need beyond in-app updates; approved change request
Asynchronous events / message queue	Measured load or failure evidence that synchronous listeners are insufficient
Pagination limits and rate limiting	Real list sizes and hosting limits
Management override of status transitions	Stakeholder confirmation that management may act on requests (M1 treats management as view/report)
Category maintenance endpoints (admin)	Team decision whether to add an FR for the M1 System Administrator need
Mature CI/CD (coverage gates, deployment)	Stable test suite and Person 2 deployment decision
Performance verification of NFR-024	M3 load/timing test on the chosen host, using the load defined under CR-001

H.4 Baseline sign-off (M2 Architecture, Technology & Initial Design Baseline)

Person 3 items in the baseline: ADR-005 to ADR-008, B.2 endpoint contract v1, repository structure (C.1), RTM v2.0 columns, Risk Register v2.0. Open items: H.3.

Team member	Role	Signature	Date
Riegardt Weich	Architecture & Quality Drivers		30 September
Ditheto Lesiba Phaho	Data, Persistence & Technology		30 September
Armand Vermeulen	Design, Development & Governance		30 September

I. References

I.1 Reference list (Harvard)

Merge with Person 1's and Person 2's lists in the PED Sources section and remove duplicates. Entries marked with an access date in square brackets need the team's own date after opening the source.

Bacchelli, A. and Bird, C. (2013) 'Expectations, outcomes, and challenges of modern code review', *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, San Francisco, 18–26 May. Piscataway, NJ: IEEE, pp. 712–721. doi:10.1109/ICSE.2013.6606617.

Beams, C. (2014) *How to write a git commit message*. Available at: https://cbea.ms/git-commit/ (Accessed: 30 September 2026).

Fielding, R.T. (2000) *Architectural styles and the design of network-based software architectures*. PhD dissertation. University of California, Irvine. Available at: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm (Accessed: 30 September 2026).

Fielding, R.T., Nottingham, M. and Reschke, J. (eds) (2022) *HTTP semantics*. RFC 9110. Internet Engineering Task Force. Available at: https://www.rfc-editor.org/info/rfc9110 (Accessed: 30 September 2026).

Fowler, M. (2002) *Patterns of enterprise application architecture*. Boston: Addison-Wesley.

Fowler, M. (2006) 'Continuous integration', *martinfowler.com*, 1 May. Available at: https://martinfowler.com/articles/continuousIntegration.html (Accessed: 30 September 2026).

Fowler, M. (2012) 'TestPyramid', *martinfowler.com*, 1 May. Available at: https://martinfowler.com/bliki/TestPyramid.html (Accessed: 30 September 2026).

Freeman, E. and Robson, E. (2020) *Head First design patterns*. 2nd edn. Sebastopol, CA: O'Reilly Media.

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) *Design patterns: elements of reusable object-oriented software*. Reading, MA: Addison-Wesley.

GitHub Docs (n.d.a) *About protected branches*. Available at: https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches (Accessed: 30 September 2026).

GitHub Docs (n.d.b) *Introduction to secret scanning*. Available at: https://docs.github.com/en/code-security/secret-scanning/introduction (Accessed: 30 September 2026).

GitHub Docs (n.d.c) *GitHub Actions documentation*. Available at: https://docs.github.com/en/actions (Accessed: 30 September 2026).

Google (n.d.) 'AIP-185: API versioning', *API Improvement Proposals*. Available at: https://google.aip.dev/185 (Accessed: 30 September 2026).

Gotel, O.C.Z. and Finkelstein, A.C.W. (1994) 'An analysis of the requirements traceability problem', *Proceedings of the First International Conference on Requirements Engineering*, Colorado Springs, 18–22 April. Los Alamitos, CA: IEEE Computer Society Press, pp. 94–101.

GraphQL Foundation (2021) *GraphQL specification, October 2021*. Available at: https://spec.graphql.org/October2021/ (Accessed: 30 September 2026).

Hohpe, G. and Woolf, B. (2003) *Enterprise integration patterns: designing, building, and deploying messaging solutions*. Boston: Addison-Wesley.

Humble, J. and Farley, D. (2010) *Continuous delivery: reliable software releases through build, test, and deployment automation*. Upper Saddle River, NJ: Addison-Wesley.

Kleppmann, M. (2017) *Designing data-intensive applications*. Sebastopol, CA: O'Reilly Media.

Martin, R.C. (2017) *Clean architecture: a craftsman's guide to software structure and design*. Upper Saddle River, NJ: Prentice Hall. (Also in Person 1's list.)

Newman, S. (2021) *Building microservices: designing fine-grained systems*. 2nd edn. Sebastopol, CA: O'Reilly Media. (Also in Person 1's list.)

Nottingham, M., Wilde, E. and Dalal, S. (2023) *Problem details for HTTP APIs*. RFC 9457. Internet Engineering Task Force. Available at: https://www.rfc-editor.org/info/rfc9457 (Accessed: 30 September 2026).

Nygard, M. (2011) 'Documenting architecture decisions', *Cognitect Blog*, 15 November. Available at: https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions (Accessed: 30 September 2026). (Also in Person 1's list.)

OpenAPI Initiative (2021) *OpenAPI specification v3.1.0*. Available at: https://spec.openapis.org/oas/v3.1.0 (Accessed: 30 September 2026).

OWASP Foundation (2023) *OWASP API security top 10 – 2023*. Available at: https://owasp.org/API-Security/editions/2023/en/0x11-t10/ (Accessed: 30 September 2026).

OWASP Foundation (n.d.a) *Logging cheat sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html (Accessed: 30 September 2026).

OWASP Foundation (n.d.b) *Secrets management cheat sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html (Accessed: 30 September 2026).

Wiggins, A. (2017) *The twelve-factor app*. Available at: https://12factor.net/ (Accessed: 30 September 2026).

American National Standards Institute/INCITS (2004), ISO/IEC (2011), Republic of South Africa (2013), Sandhu, Ferraiolo and Kuhn (2000), Bass, Weber and Zhu (2015).

Bass, L., Clements, P. and Kazman, R. (2021) *Software architecture in practice*. 4th edn. Boston: Addison-Wesley.

Brown, S. (n.d.) *The C4 model for visualising software architecture*. Available at: https://c4model.com (Accessed: [date]).

Evans, E. (2004) *Domain-driven design: tackling complexity in the heart of software*. Boston: Addison-Wesley.

Fowler, M. (2002) *Patterns of enterprise application architecture*. Boston: Addison-Wesley.

Fowler, M. (2015) 'MonolithFirst', *martinfowler.com*, 3 June. Available at: https://martinfowler.com/bliki/MonolithFirst.html (Accessed: [date]).

Kruchten, P. (1995) 'The 4+1 view model of architecture', *IEEE Software*, 12(6), pp. 42–50.

Martin, R.C. (2017) *Clean architecture: a craftsman's guide to software structure and design*. Upper Saddle River, NJ: Prentice Hall.

Newman, S. (2021) *Building microservices: designing fine-grained systems*. 2nd edn. Sebastopol, CA: O'Reilly Media.

Nygard, M. (2011) 'Documenting architecture decisions', *Cognitect Blog*, 15 November. Available at: https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions (Accessed: [date]).

OWASP Foundation (2021) *OWASP Top 10:2021, A01 Broken Access Control*. Available at: https://owasp.org/Top10/A01_2021-Broken_Access_Control/ (Accessed: [date]).

Richards, M. and Ford, N. (2020) *Fundamentals of software architecture: an engineering approach*. Sebastopol, CA: O'Reilly Media.